Récursivité partie 1

Term NSI — Nov 2026

fonction récursive, écriture à partir d'une fonction récurrente, preuves

1 Récursivité — partie 1 : principe et preuve

Prérequis (TD « Invariants de boucle et raisonnement par récurrence ») - écrire une boucle à partir d'une **relation de récurrence** (multiplication, factorielle, division euclidienne, Horner) ; - prouver une boucle avec un **invariant** (initialisation, conservation, conclusion) ; - prouver la terminaison d'une boucle `while` avec un **variant**.

Objectifs de ce cours - écrire une fonction **récursive** à partir d'une relation de récurrence ; - comprendre son exécution grâce à la **pile d'appels** ; - prouver qu'une fonction récursive **termine** et qu'elle est **correcte** ; - appliquer la récursivité aux nombres, puis aux **chaînes** et aux **listes** (préparation du TP).

---

2 1. Point de départ : la question de fin de TD

À la fin du TD, la question était : *« Imaginons qu'une fonction Python puisse s'appeler elle-même pour obtenir le résultat au rang $k-1$. Écrire `factorielle(n)` sans boucle. »*

La relation de récurrence de la factorielle est :

$$\begin{cases} 0! = 1 \\ n! = n \times (n-1)!, \text{pour} \quad n \geq 1 \end{cases}$$

Version itérative (TD, exercice 5)

```
def factorielle_iter(n):
    f = 1
    for k in range(1, n + 1):
        f = f * k
    return f
```

Version récursive

```
def factorielle(n):
    """renvoie n! pour un entier n >= 0
    >>> factorielle(5)
    120
    """
    if n == 0:                          # cas de base   : 0! = 1
        return 1
    else:
        return n * factorielle(n - 1)   # cas récursif  : n! = n × (n-1)!
```

La version récursive est une **traduction directe** de la relation de récurrence : chaque ligne de la définition mathématique devient une ligne du programme. Il n'y a ni accumulateur, ni compteur de boucle.

---

3 2. Définition

Un algorithme est dit **récursif** s'il résout un problème en se ramenant à **une ou plusieurs instances du même problème portant sur des données plus petites**. En pratique, une fonction récursive est une fonction qui **s'appelle elle-même** dans sa propre définition.

Cette simple autoréférence pourrait faire croire que la fonction va s'appeler indéfiniment. Ce n'est pas le cas, car une fonction récursive correcte contient toujours deux ingrédients :

- un **cas de base** : une (ou plusieurs) valeur(s) de l'argument pour laquelle le résultat est connu **immédiatement**, sans appel récursif ;
- un **cas récursif** (ou *hérédité*) : un appel de la fonction avec un argument **plus petit**, c'est-à-dire plus proche du cas de base.

À chaque appel, les arguments changent : les calculs ne sont donc pas tous identiques. Une valeur est calculée au cas de base, puis **remonte** d'appel en appel jusqu'au premier.

Schéma général

```
def fonction_recursive(param):
    if <cas de base>:
        return <valeur connue>
    else:
        # éventuellement d'autres instructions
        return <calcul utilisant fonction_recursive(param plus petit)>
```

Les questions à se poser pour élaborer l'algorithme

- De quel(s) paramètre(s) dépend le problème ? Qu'est-ce qui mesure sa « taille » ?
- Sait-on résoudre le problème directement lorsque cette taille est « petite » ? ⟶ **cas de base**
- Sait-on résoudre le problème en supposant qu'on connaît la solution pour une taille plus petite ? ⟶ **cas récursif**

C'est exactement la démarche du TD : une relation de récurrence donne un **cas de départ** et une formule qui exprime le rang `k` à partir du rang `k - 1`.

---

4 3. Exemple détaillé : multiplier sans `*`

4.1 3.1 La relation de récurrence (TD, exercice 4)

Pour un entier `a` et un entier `b ≥ 0` :

$$\begin{cases} a \times 0 = 0 \\ a \times b = a \times (b-1) + a, \text{pour} \quad b \geq 1 \end{cases}$$

Au TD, on l'a programmée avec une boucle `for` et une variable `p` qui accumule le résultat. La version récursive est :

```
def multiplie(a, b):
    """renvoie a × b, pour b entier >= 0, en n'utilisant que l'addition
    >>> multiplie(7, 4)
    28
    """
    if b == 0:
        return 0
    else:
        return multiplie(a, b - 1) + a
```

Soit $(u_n)_{n\in\mathbb{N}}$ la suite définie par: $\begin{cases} u_0 = 0 \\ \forall n \in \mathbb{N} \quad u_{n+1} = u_n + 2 \end{cases}$

Fig. 1. – suite récurrente

4.2 3.2 Exécution : la pile d'appels

Lors de l'appel `multiplie(7, 4)`, la fonction ne peut pas calculer `multiplie(7, 3) + 7` tant qu'elle ne connaît pas `multiplie(7, 3)`. Elle se met donc **en attente** et lance un nouvel appel, et ainsi de suite jusqu'au cas de base :

```
multiplie(7, 4) = multiplie(7, 3) + 7
    multiplie(7, 3) = multiplie(7, 2) + 7
        multiplie(7, 2) = multiplie(7, 1) + 7
            multiplie(7, 1) = multiplie(7, 0) + 7
                multiplie(7, 0) = 0                  ← cas de base
            multiplie(7, 1) = 0 + 7  = 7
        multiplie(7, 2) = 7 + 7  = 14
    multiplie(7, 3) = 14 + 7 = 21
multiplie(7, 4) = 21 + 7 = 28
```

Python mémorise les appels en attente dans une **pile d'appels** (structure LIFO : le dernier appel arrivé est le premier terminé). Chaque appel possède **son propre contexte** : ses propres variables locales `a` et `b`.

Phase 1 — empilement : chaque nouvel appel est placé au sommet de la pile.

```
                                                | multiplie(7,0) |
                              | multiplie(7,1) | | multiplie(7,1) |
                  ...         | multiplie(7,2) | | multiplie(7,2) |
| multiplie(7,4) |            | multiplie(7,3) | | multiplie(7,3) |
                              | multiplie(7,4) | | multiplie(7,4) |
```

Phase 2 — dépilement : le sommet `multiplie(7, 0)` renvoie `0` sans nouvel appel. Il est retiré de la pile et sa valeur est transmise à l'appel situé juste en dessous, qui peut alors terminer son calcul, et ainsi de suite.

```
| multiplie(7,0) | → 0
| multiplie(7,1) |      | multiplie(7,1) | → 7
| multiplie(7,2) |      | multiplie(7,2) |      | multiplie(7,2) | → 14
| multiplie(7,3) |      | multiplie(7,3) |      | multiplie(7,3) |      ...
| multiplie(7,4) |      | multiplie(7,4) |      | multiplie(7,4) |
```

À la fin, la pile est vide et `multiplie(7, 4)` renvoie `28`.

> **Lien avec le TD.** Les valeurs renvoyées pendant le dépilement (0, 7, 14, 21, 28) sont **exactement** celles de la colonne `p` du tableau de suivi de l'exercice 4. La boucle et la récursion calculent les mêmes termes de la suite, mais : - la boucle part de `p = 0` et monte jusqu'au résultat ; - la fonction récursive part de `b = 4`, **descend** jusqu'au cas de base, puis les résultats **remontent**.

{{< img src=« ../images/page2/pythontutor.GIF » alt=« animation recursivité additions successives » caption=« animation issue de l'application pythontutor » >}}

Conseil : visualiser vos propres fonctions récursives avec pythontutor pour observer la pile.

4.3 3.3 Les limites de la pile : `RecursionError`

La pile d'appels occupe de la mémoire. Pour se protéger, Python limite sa hauteur (environ **1000** appels par défaut) :

```
>>> multiplie(7, 5000)
RecursionError: maximum recursion depth exceeded
```

On peut consulter ou modifier cette limite avec le module `sys` (`sys.getrecursionlimit()`, `sys.setrecursionlimit(n)`), mais cela reste à utiliser avec prudence.

La même erreur apparaît lorsque le **cas de base est oublié** ou **jamais atteint** :

```
def factorielle_fausse(n):
    return n * factorielle_fausse(n - 1)   # pas de cas de base !
```

`factorielle_fausse(3)` appelle `factorielle_fausse(2)`, `(1)`, `(0)`, `(-1)`, `(-2)`... sans jamais s'arrêter : c'est une **récursion infinie**, interrompue par Python avec une `RecursionError`.

---

5 4. Prouver un algorithme récursif

Au TD, pour prouver une boucle, on a utilisé un **invariant** `P(k)` vrai après `k` tours, démontré en trois étapes. Pour une fonction récursive, il n'y a pas de boucle donc pas d'invariant : on démontre **directement** une propriété de la fonction **par récurrence sur la taille de l'argument**.

	Algorithme itératif (TD)	Algorithme récursif
Propriété à prouver	invariant `P(k)` : « après `k` tours, ... »	`P(n)` : « l'appel `f(n)` renvoie ... »
Étape 1	**Initialisation** : `P(0)` avant le 1er tour	**Cas de base** : `P(0)` est vraie
Étape 2	**Conservation** : si `P(k)` au début d'un tour, alors `P(k+1)` à la fin	**Hérédité** : si l'appel récursif `f(n−1)` est correct, alors `f(n)` est correct
Étape 3	**Conclusion** : à la sortie de la boucle	**Conclusion** : `P(n)` est vraie pour tout `n`
Terminaison	variant (boucle `while`)	variant : l'argument qui décroît strictement à chaque appel

5.1 4.1 Terminaison : un variant

> **À retenir.** Une fonction récursive **termine** si l'on trouve un **variant** : une quantité **entière**, **positive**, qui **décroît strictement** à chaque appel récursif, et telle que lorsqu'elle atteint sa plus petite valeur, on est dans un cas de base.

Exemple : `multiplie(a, b)`. Le variant est `b` :

- `b` est un entier, `b ≥ 0` (c'est une **précondition** de la fonction) ;
- chaque appel `multiplie(a, b)` avec `b ≥ 1` appelle `multiplie(a, b − 1)` : `b` décroît strictement ;
- lorsque `b = 0`, on est dans le cas de base : aucun nouvel appel.

Une suite d'entiers positifs strictement décroissante ne peut pas être infinie : la fonction termine, après exactement `b` appels récursifs.

Attention aux préconditions : `multiplie(7, -1)` appelle `multiplie(7, -2)`, `multiplie(7, -3)`... `b` reste un variant décroissant, mais il n'est plus positif et le cas de base n'est jamais atteint. C'est la même situation que `division(17, 0)` au TD : la preuve n'est valable que si la précondition est respectée. On peut la vérifier dans le code avec `assert b >= 0`.

5.2 4.2 Correction : exemple rédigé (modèle à suivre)

> **Propriété** `P(n)` : « `factorielle(n)` renvoie `n!` ».

- **Cas de base** (`n = 0`) : le test `n == 0` est vrai, la fonction renvoie `1`, et `0! = 1`. Donc `P(0)` est vraie.

- **Hérédité** : soit `n ≥ 1`. Supposons `P(n − 1)` vraie, c'est-à-dire que `factorielle(n − 1)` renvoie `(n − 1)!`. Comme `n ≠ 0`, `factorielle(n)` renvoie `n * factorielle(n − 1)`, c'est-à-dire, d'après l'hypothèse de récurrence, $n \times (n-1)! = n!$. Donc `P(n)` est vraie.
- **Conclusion** : par récurrence, `factorielle(n)` renvoie `n!` pour tout entier `n ≥ 0`.

Terminaison : le variant est `n` (entier, positif, diminue de 1 à chaque appel, cas de base en `n = 0`). La fonction fait `n` appels récursifs.

> **Remarque essentielle.** Dans l'hérédité, on **fait confiance** à l'appel récursif : on suppose qu'il renvoie le bon résultat, sans dérouler toute la pile. C'est aussi la bonne manière de **concevoir** une fonction récursive : *« si je sais résoudre le problème pour une taille plus petite, comment j'en déduis la solution pour la taille actuelle ? »*

À faire : rédiger de la même façon la preuve de correction de `multiplie(a, b)`, avec la propriété `P(b)` : « `multiplie(a, b)` renvoie `a × b` ».

5.3 4.3 Quand l'argument diminue de plus de 1 : la division euclidienne

On reprend la relation du TD (exercice 6) : pour `a ≥ 0` et `b > 0`, si `a < b` le quotient vaut `0` et le reste `a` ; sinon, `quotient(a, b) = 1 + quotient(a − b, b)` et `reste(a, b) = reste(a − b, b)`.

```
def division(a, b):
    """renvoie le couple (quotient, reste) de la division de a par b
    a >= 0 et b > 0
    >>> division(17, 5)
    (3, 2)
    """
    if a < b:
        return (0, a)
    q, r = division(a - b, b)
    return (q + 1, r)
```

Pile d'appels : `division(17, 5)` ⟶ `division(12, 5)` ⟶ `division(7, 5)` ⟶ `division(2, 5)` renvoie `(0, 2)`, puis `(1, 2)`, `(2, 2)` et enfin `(3, 2)`.

- **Terminaison** : le variant est `a`. C'est un entier positif ; comme `b > 0`, `a − b < a` : il décroît strictement. Et il ne peut pas « sauter » le cas de base : dès que `a < b`, on s'arrête. Si `b = 0`, `a − b = a` ne décroît plus : récursion infinie.
- **Correction** : on suppose que l'appel `division(a − b, b)` renvoie un couple `(q, r)` avec `a − b = b × q + r` et `0 ≤ r < b`. Alors `a = b × (q + 1) + r` : le couple `(q + 1, r)` est bien le quotient et le reste de la division de `a` par `b`.

Ici l'hypothèse de récurrence ne porte pas sur `a − 1` mais sur `a − b` : on suppose la propriété vraie **pour toutes les valeurs plus petites** que `a` (on parle de *récurrence forte*). Le principe reste le même.

---

6 5. Récursivité sur les séquences : chaînes et listes

La récursivité ne s'applique pas qu'aux nombres. Pour une chaîne ou une liste, la « taille » est sa **longueur** : on réduit le problème en retirant un élément.

On utilise les **tranches** (*slices*) déjà rencontrées au TD avec la notation `t[:k]` :

Expression	Signification	Exemple avec `t = [3, 1, 4]`
`t[0]`	premier élément	`3`
`t[1:]`	tout sauf le premier élément	`[1, 4]`
`t[-1]`	dernier élément	`4`
`t[:-1]`	tout sauf le dernier élément	`[3, 1]`

Les mêmes notations fonctionnent pour une chaîne : `"lustucru"[1:]` vaut `"ustucru"`.

6.1 5.1 Découpage « premier élément + reste » : somme d'une liste

Relation de récurrence : la somme d'une liste vide vaut 0 ; sinon, `somme(t) = t[0] + somme(t[1:])`.

```
def somme(t):
    """renvoie la somme des éléments de la liste t
    >>> somme([3, 1, 4])
    8
    """
    if len(t) == 0:          # cas de base : liste vide
        return 0
    return t[0] + somme(t[1:])
```

```
somme([3, 1, 4]) = 3 + somme([1, 4])
                 = 3 + 1 + somme([4])
                 = 3 + 1 + 4 + somme([])
                 = 3 + 1 + 4 + 0
                 = 8
```

Variant : la longueur de `t`, qui diminue de 1 à chaque appel jusqu'à 0.

6.2 5.2 Découpage « début + dernier élément » : la méthode de Horner

Au TD (exercice 7), on a établi : valeur de `""` = 0, et valeur de `s + c` = 2 × valeur de `s` + `c`. Cette relation porte sur le **dernier** caractère, on le retire donc avec `s[:-1]` :

```
def binaire_vers_decimal(s):
    """s : chaîne de '0' et de '1', bit de poids fort en premier
    >>> binaire_vers_decimal("1011")
    11
    """
    if s == "":
        return 0
    return 2 * binaire_vers_decimal(s[:-1]) + int(s[-1])
```

```
binaire_vers_decimal("1011") = 2 × binaire_vers_decimal("101") + 1
                                 = 2 × (2 × binaire_vers_decimal("10") + 1) + 1
                                 ...  = 2 × 5 + 1 = 11
```

6.3 5.3 Le cas de base n'est pas toujours « vide » : maximum d'une liste

Le maximum d'une liste vide n'a pas de sens : le cas de base est ici une liste à **un seul** élément, et l'hérédité contient un **test**.

```
def maximum(t):
    """renvoie le plus grand élément d'une liste NON VIDE t
    >>> maximum([3, 1, 4, 1, 5])
    5
    """
    if len(t) == 1:
        return t[0]
    m = maximum(t[1:])       # maximum du reste de la liste
    if t[0] > m:
        return t[0]
    else:
        return m
```

7 6. Plusieurs cas de base, plusieurs appels récursifs

7.1 6.1 Plusieurs cas de base

Une fonction peut avoir plusieurs cas de base. Il faut alors vérifier que **toute chaîne d'appels finit par atteindre l'un d'eux, quelle que soit la valeur de départ autorisée**. Question à se poser systématiquement : *« l'argument peut-il passer à côté du cas de base ? »* (voir TP, exercice 1).

7.2 6.2 Récursivité multiple : la suite de Fibonacci

*La suite de Fibonacci est une suite récurrente d'ordre 2 : chaque terme se calcule à l'aide des **deux** termes précédents.*

Fibonacci était le surnom de Léonard de Pise (1170-1250). Il cherchait le nombre de couples de lapins au bout de `n` années, lorsque :

- lors de la première année, l'éleveur démarre avec un couple de lapins nouveau-nés ;
- un couple donne naissance à un nouveau couple tous les ans, à partir de sa 2e année.

année	couples de l'année précédente	nouveau nombre de couples
0	0	0
1	0	1
2	1	1 + 0 = 1
3	1	1 + 1 = 2
4	2	2 + 1 = 3
5	3	3 + 2 = 5
6	…	…

{{< img src=« ../images/page2/fibo2.png » >}}

$$\begin{cases} f_0 = 0 \\ f_1 = 1 \\ f_{n+1} = f_n + f_{n-1}, \forall n \in \mathbb{N}^* \end{cases}$$

Les nombres de Fibonacci apparaissent aussi dans la croissance des plantes : le nombre de pétales de nombreuses fleurs est un nombre de Fibonacci, et le rapport de deux termes successifs se rapproche du **nombre d'or**.

Cette définition comporte **deux cas de base** et le cas récursif fait **deux appels**. L'exécution ne forme plus une simple pile linéaire, mais un **arbre d'appels** : la fonction écrite au TP (exercice 2) appelle par exemple plusieurs fois `fibo(2)` pour calculer `fibo(4)`.

{{< img src=« ../images/page2/fibo.png » caption=« arbre d'appels pour la suite de Fibonacci récursive » >}}

Ces calculs répétés posent un problème d'efficacité, étudié dans la partie 2 (complexité).

7.3 6.3 Plusieurs paramètres qui évoluent : l'algorithme d'Euclide

Dans certaines récursions, ce sont **deux** paramètres qui changent à chaque appel. C'est le cas de l'algorithme d'Euclide pour le PGCD, présenté et programmé dans le TP (exercice 6).

8 7. Itératif ou récursif ?

Tout algorithme récursif peut s'écrire de manière itérative, et réciproquement. L'une ou l'autre forme peut paraître plus naturelle suivant le problème.

	Itératif	Récursif
Principe	une boucle répète un bloc d'instructions	la fonction s'appelle elle-même sur une donnée plus petite
Ce qu'il faut écrire	initialisation, condition de boucle, mise à jour des variables	cas de base et cas récursif
Lien avec la récurrence	les variables accumulent les termes successifs	traduction directe de la relation de récurrence
Terminaison	`for` : toujours ; `while` : variant	variant sur l'argument
Correction	invariant de boucle	récurrence sur la taille de l'argument
Mémoire	quelques variables	une pile d'appels qui grandit (limite ≈ 1000 en Python)
Lisibilité	parfois plus longue	souvent plus courte et proche de la définition

---

9 8. Méthode : écrire une fonction récursive

À retenir

1. **Taille du problème** : identifier le paramètre qui va diminuer (un entier, la longueur d'une chaîne ou d'une liste…).
2. **Cas de base** : traiter directement la (les) plus petite(s) valeur(s).
3. **Cas récursif** : *supposer* que la fonction est correcte pour une taille plus petite, et en déduire le résultat pour la taille actuelle.
4. **Terminaison** : vérifier que l'argument se rapproche strictement du cas de base, et qu'il ne peut pas le « sauter » (préconditions).
5. **Tester** : utiliser des `assert` sur de petits cas, et dérouler la pile à la main en cas de doute.

---

10 Vers le TP

Le TP met en pratique ce cours :

Exercice du TP	Notion utile dans ce cours
1. `impair`	cas de base, variant qui diminue de 2 (§ 4.3, § 6.1)
2. Fibonacci	récursivité multiple (§ 6.2)
3. Tours de Hanoï	« faire confiance » à l'appel récursif (§ 4.2) — approfondi en partie 2
4. `nombre_r`	récursivité sur une chaîne, découpage `phrase[1:]`, test dans l'hérédité (§ 5.1, § 5.3)
5. `reverse_recur`	récursivité sur une chaîne (§ 5)
6. PGCD d'Euclide	deux paramètres qui évoluent (§ 6.3)

Partie 2 : complexité des algorithmes récursifs (factorielle, Fibonacci, Euclide), puis applications moins mathématiques (tours de Hanoï…).